CS486 – Introduction to Databases

LAB
Topic 10: Database Application

Lecturer: Lê Thị Nhàn, PhD.
Lab instructor: Nguyễn Ngọc Toàn, MSc.
Nguyễn Ngọc Minh Châu, MSc.

1. Pre-setup

- In this tutorial, you will need the following tools/frameworks/ libraries to execute the script: **(Install them before the class)**
  + Python, Jupiter Notebook
  + Python dependency: pyodbc, pandas, ipykernel
  + ODBC driver

a) Jupyter Notebook installation

Follow the following steps to install the Jupyter Notebook after you downloaded and installed Python from https://www.python.org/:

1. Open a terminal and check whether python and pip are installed by running the following script:

```
python  - -version
pip  - - version
```

2. Install Python dependencies:

```
pip  install pyodbc pandas ipykernel
```

b) ODBC driver installation

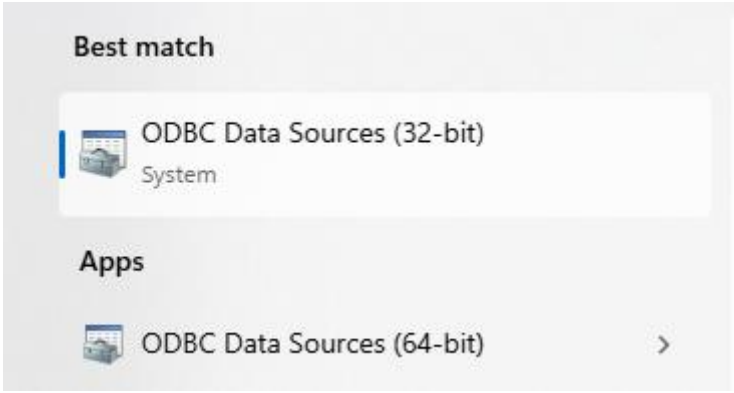
- In order to create the connection to the database from our application, we need a driver. A driver is the software component that allows your application to communicate with a specific database system. The driver acts as a translator, converting your application's requests into a format that the database understands and vice versa. Follow the instructions below to install ODBC:

On Windows: [link]

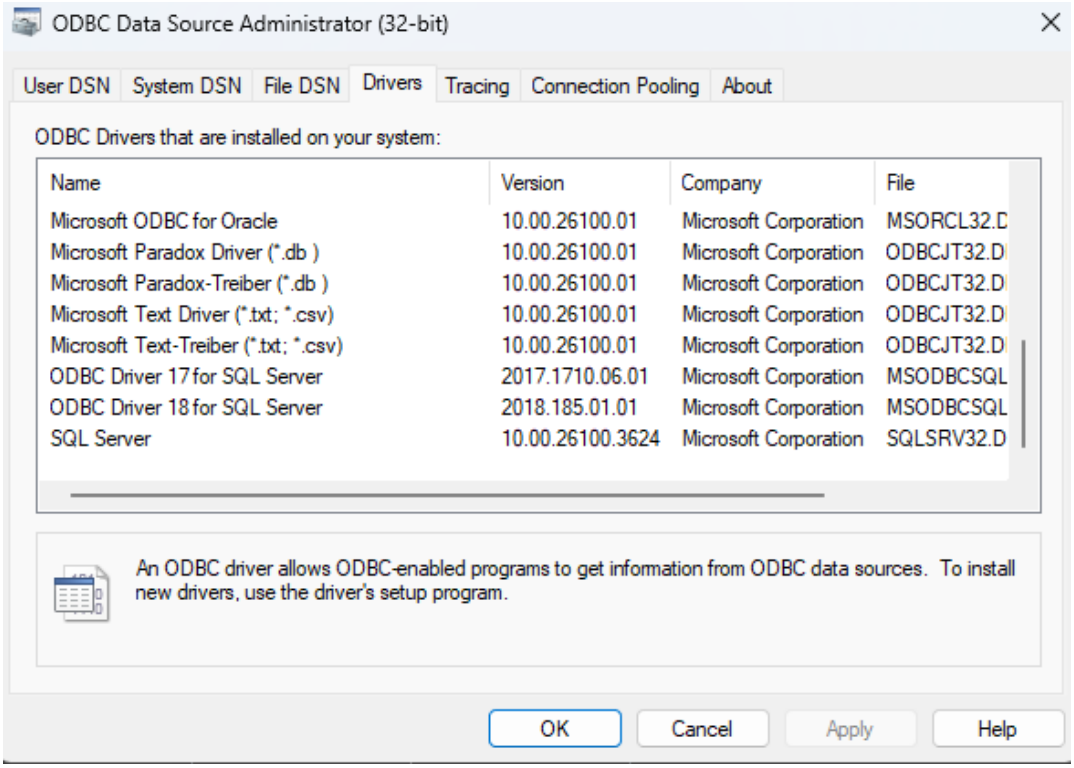
On Linux: [link]

On MacOS: [link]

- How to check ODBC versions on **Windows**:
  1. Type “ODBC Data Source” on the search bar to open ODBC Data Source Administrator:



2. Navigate to Drivers tab and check for ODBC Driver xx for SQL Server:

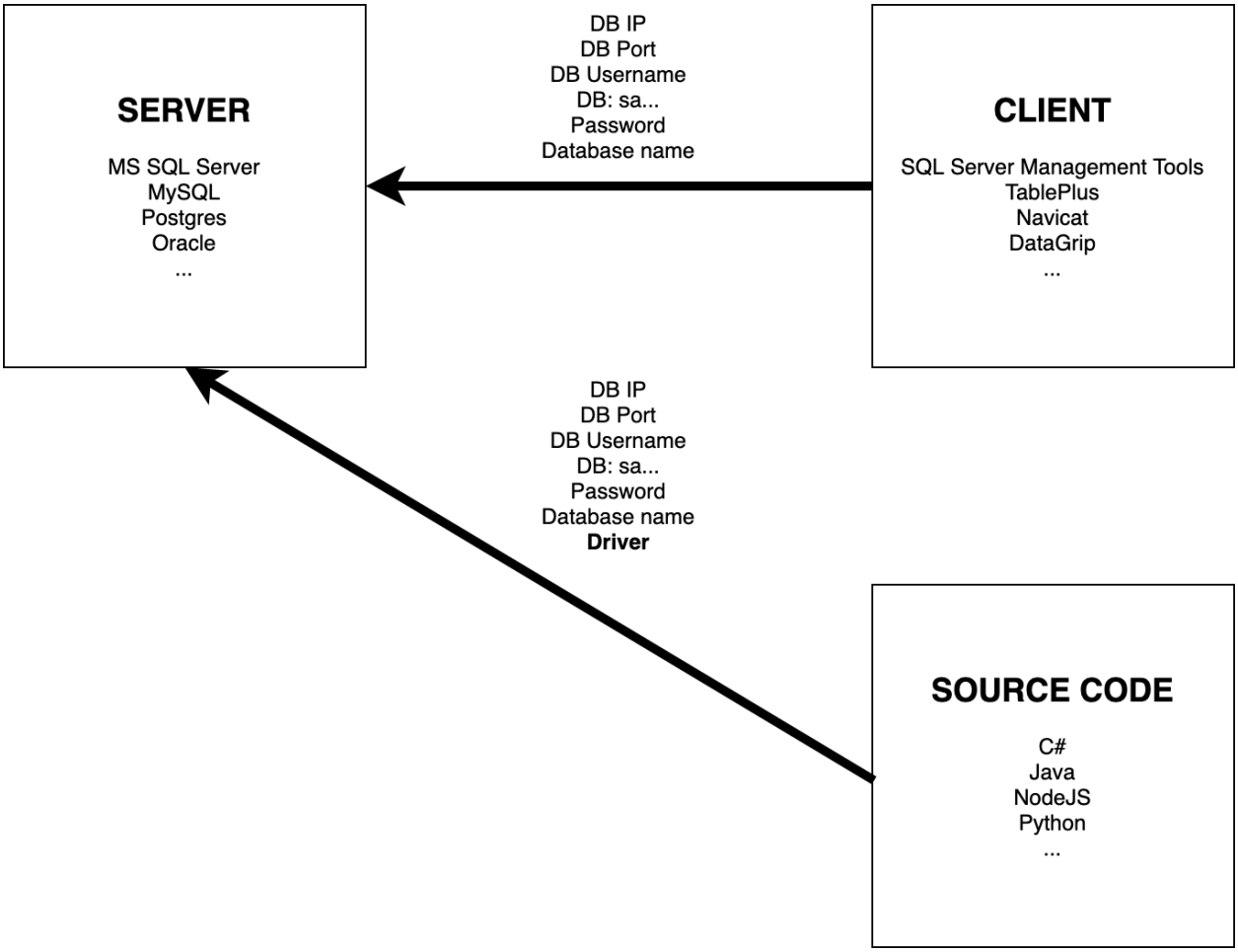


For the above example, we can see that ODBC Driver version 17 and 18 are installed in the device.

2. Database Application

In this tutorial, we will create a sample python application which can interact with our University database created in SSMS.

a) Create database connection

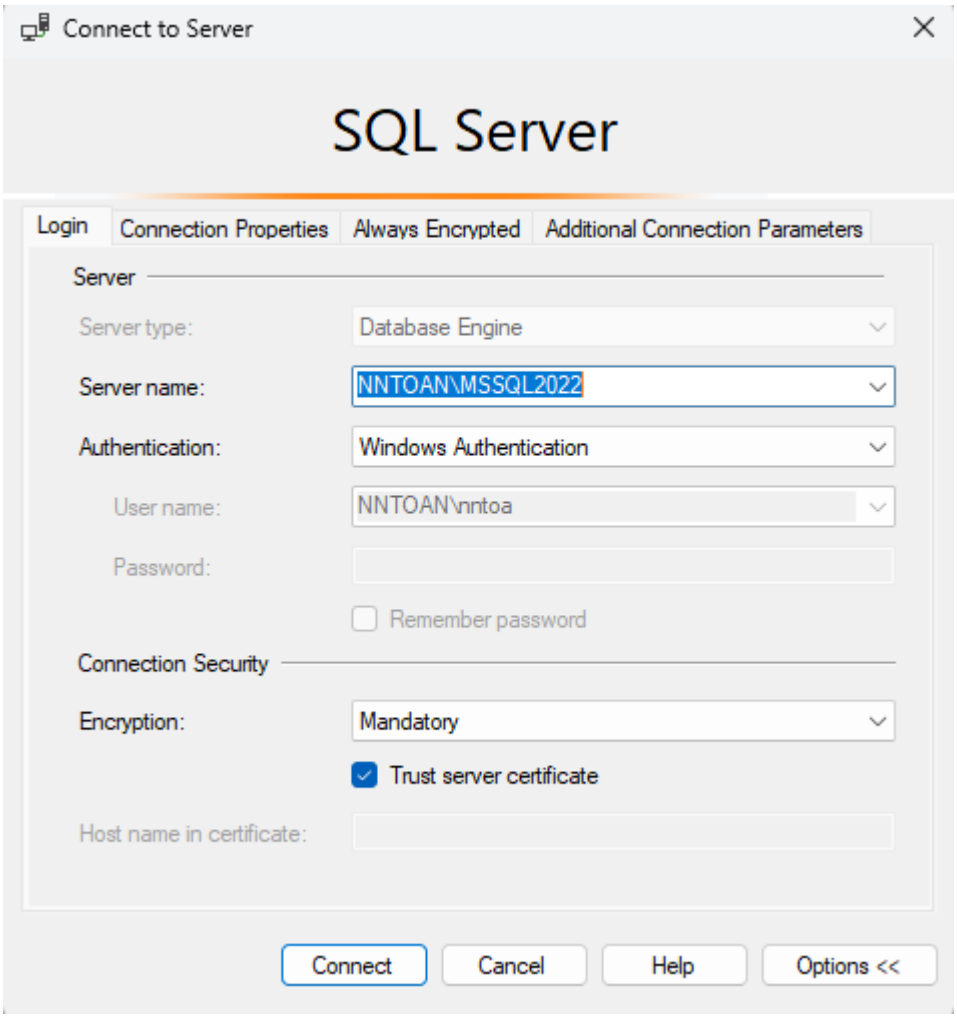


Step 1: We want to create a **connection string** (All options available on this link):

```
# Example connection string for a local SQL Server instance using Password authentication
CONNECTION_STRING = (
    "Driver={ODBC Driver 18 for SQL Server};"
    "Server=localhost;"
    "Database=University;"
    "UID=<User_ID>;"
    "PWD=<Password>;"
    "TrustServerCertificate=yes;"
)
```

Note:

- For **Windows Authentication** login type, we simply add "Trusted_Connection=yes;" to the connection string and remove the UID, PWD strings.
- All the field values are similar to the connection profile of SSMS or vscode database connection, you can fill in the current connection profile's value to the connection string:



Step 2: We use pyodbc library to connect to our database using the connection string in Step 1:

```
def get_db_connection():
    """Establishes and returns a connection to the database."""
    try:
        conn = pyodbc.connect(CONNECTION_STRING)
        return conn
    except pyodbc.Error as ex:
        print(f"Database connection error: {ex}")
        return None
```

b) Database operation examples

Imagining we have a mobile/web application which uses HTTPs methods to perform the basic operations of **CRUD** (Create, Read, Update, Delete) on database records.

- **Example 1:** Read all student data

```
conn = get_db_connection()
if conn:
    cursor = conn.cursor()
    try:
        cursor.execute("SELECT student_id, student_name, gender, birthdate, class, department_id FROM Student")
        columns = [column[0] for column in cursor.description]
        students_data = cursor.fetchall()

        # Display the results in a formatted way
        df = pd.DataFrame.from_records(students_data, columns=columns)
        print("All Students:")
        print(df)

    except pyodbc.Error as ex:
        print(f"Error fetching data: {ex}")
    finally:
        conn.close()
```

- **Example 2:** Add a new student

```
new_student_data = {
    "student_id": "ST021",
    "student_name": "John Doe",
    "gender": "M",
    "birthdate": "2003-01-01",
    "class": "23TT1",
    "department_id": "CS"
}

conn = get_db_connection()
if conn:
    cursor = conn.cursor()
    try:
        # Using a parameterized query to prevent SQL Injection
        query = "INSERT INTO Student (student_id, student_name, gender, birthdate, class, department_id) VALUES (?, ?, ?, ?, ?, ?)"
        cursor.execute(
            query,
            (
                new_student_data['student_id'],
                new_student_data['student_name'],
                new_student_data['gender'],
                new_student_data['birthdate'],
                new_student_data['class'],
                new_student_data['department_id']
            )
        )
        conn.commit()
        print(f"Student {new_student_data['student_id']} added successfully.")
    except pyodbc.Error as ex:
        conn.rollback()
        print(f"Error adding student: {ex}")
    finally:
        conn.close()
```

- **Example 3:** Update an existing Student

```python
# The student ID to update
student_id_to_update = "ST001"

# The fields to change
updates = {
    "department_id": "AI",
    "class": "24TT2"
}

conn = get_db_connection()
if conn:
    cursor = conn.cursor()
    try:
        updates_sql = ", ".join([f"{key} = ?" for key in updates.keys()])
        print(f"Updates SQL: {updates_sql}")
        params = list(updates.values())
        params.append(student_id_to_update)

        query = f"UPDATE Student SET {updates_sql} WHERE student_id = ?"
        cursor.execute(query, tuple(params))
        conn.commit()

        if cursor.rowcount > 0:
            print(f"Student {student_id_to_update} updated successfully.")
        else:
            print(f"No student found with id {student_id_to_update}.")

    except pyodbc.Error as ex:
        conn.rollback()
        print(f"Error updating student: {ex}")
    finally:
        conn.close()
```

- **Example 4:** Delete a student

```python
# The student ID to delete
student_id_to_delete = "ST021"

conn = get_db_connection()
if conn:
    cursor = conn.cursor()
    try:
        # Delete related records from GradeReport first to satisfy foreign key constraints
        cursor.execute("DELETE FROM GradeReport WHERE student_id = ?", student_id_to_delete)
        # Then delete the student record
        cursor.execute("DELETE FROM Student WHERE student_id = ?", student_id_to_delete)
        conn.commit()

        if cursor.rowcount > 0:
            print(f"Student {student_id_to_delete} and associated records deleted successfully.")
        else:
            print(f"No student found with id {student_id_to_delete}.")

    except pyodbc.Error as ex:
        conn.rollback()
        print(f"Error deleting student: {ex}")
    finally:
        conn.close()
```

Note: Cursors represent a database cursor, which is used to manage the context of a fetch operation. Cursors created from the same connection are **not isolated**, i.e., any changes done to the database by a cursor are immediately visible by the **other** cursors.